

Programming Language Principles, Design, and Implementation

Lecture 9 - Polymorphism

Sean Moss

University of Birmingham

Where are we?

- ▶ **Part 1: Principles of programming**
 - ▶ Functional languages
 - ▶ Operational semantics
 - ▶ **Type systems**
 - ▶ Abstract datatypes
- ▶ Part 2: Compilers

Today

Polymorphism

- ▶ Let-Polymorphism
- ▶ System F

Further reading

- ▶ “Types and Programming Languages”, Pierce, Chapters 22 & 23

Recap: Type Checking vs. Type Inference

The Church-style Simply Typed λ -Calculus

Syntax:

$$\begin{aligned} T &::= \alpha \mid \mathbb{B} \mid T \rightarrow T \\ M &::= x \mid \lambda x : T. M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M \end{aligned}$$

The Curry-style Simply Typed λ -Calculus

Syntax:

$$\begin{aligned} T &::= \alpha \mid \mathbb{B} \mid T \rightarrow T \\ M &::= x \mid \lambda x. M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M \end{aligned}$$

- ▶ We saw type checkers for both calculi.
- ▶ Could types be automatically inferred?

Recap: Type Checking vs. Type Inference

Type checking: process of checking that some given **expression** has some given **type** w.r.t. given **typing rules**

Syntax directed rules of the Curry-style Simply-Typed λ -Calculus:

$$\frac{}{\Gamma, x : T \vdash x : T} \text{VAR} \qquad \frac{\Gamma, x : T \vdash M : U}{\Gamma \vdash \lambda x. M : T \rightarrow U} \text{ABS}$$

$$\frac{\Gamma \vdash M : T \rightarrow U \quad \Gamma \vdash N : T}{\Gamma \vdash M N : U} \text{APP}$$

$$\frac{}{\Gamma \vdash \text{true} : \mathbb{B}} \text{T} \qquad \frac{}{\Gamma \vdash \text{false} : \mathbb{B}} \text{F}$$

$$\frac{\Gamma \vdash M : \mathbb{B} \quad \Gamma \vdash N : T \quad \Gamma \vdash P : T}{\Gamma \vdash \text{if } M \text{ then } N \text{ else } P : T} \text{ITE}$$

Type inference: process of inferring the type(s) of an expression without type annotations so that it type checks according to given typing rules.

We can use **Wand's algorithm** for type inference.

Polymorphism

For more interesting examples, let's extend our (Curry-style) language with numbers and let-expressions.

Syntax:

$$\begin{aligned} T &::= \alpha \mid \mathbb{B} \mid \mathbb{N} \mid T \rightarrow T \\ M &::= x \mid \lambda x.M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M \\ &\quad \mid \text{let } x = M \text{ in } M \\ &\quad \mid \text{zero} \mid \text{succ } M \mid \text{pred } M \mid \text{iszero } M \end{aligned}$$

Values: $V ::= \lambda x.M \mid \text{true} \mid \text{false} \mid \text{zero} \mid \text{succ } V$

Call-by-value evaluation contexts

$$\begin{aligned} C &::= \bullet \mid C M \mid V C \mid \text{if } C \text{ then } M \text{ else } M \\ &\quad \mid \text{let } x = C \text{ in } M \mid \text{pred } C \mid \text{iszero } C \mid \text{succ } C \end{aligned}$$

Polymorphism

Call-by-value small-step operational semantics rules:

$$\frac{}{(\lambda x.M) V \longrightarrow_v M[x \setminus V]} \beta$$

$$\frac{M \longrightarrow_v N}{C[M] \longrightarrow_v C[N]} \text{CTX}_C$$

$$\frac{}{\text{if true then } M \text{ else } N \longrightarrow_v M} \text{IteT}$$

$$\frac{}{\text{if false then } M \text{ else } N \longrightarrow_v N} \text{IteF}$$

$$\frac{}{\text{let } x = V \text{ in } M \longrightarrow_v M[x \setminus V]} \text{LetV}$$

$$\frac{M \longrightarrow_v N}{\text{succ } M \longrightarrow_v \text{succ } N} \text{Succ}$$

$$\frac{}{\text{pred zero} \longrightarrow_v \text{zero}} \text{PredZ}$$

$$\frac{}{\text{pred (succ } V) \longrightarrow_v V} \text{PredS}$$

$$\frac{}{\text{iszero zero} \longrightarrow_v \text{true}} \text{IsZZ}$$

$$\frac{}{\text{iszero (succ } V) \longrightarrow_v \text{false}} \text{IsZS}$$

Polymorphism

New typing rules:

$$\begin{array}{c} \frac{}{\Gamma \vdash \text{zero} : \mathbb{N}} \text{Z} \qquad \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \text{succ } M : \mathbb{N}} \text{S} \qquad \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \text{pred } M : \mathbb{N}} \text{P} \\[1em] \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \text{iszero } M : \mathbb{B}} \text{I} \qquad \frac{\Gamma \vdash M : T \quad \Gamma, x : T \vdash N : U}{\Gamma \vdash \text{let } x = M \text{ in } N : U} \text{LET} \end{array}$$

Let's define `one` as `succ zero`

Is the following “reasonable” expression well-typed?

$(\lambda f. \text{if } (f \text{ true}) \text{ then } (f \text{ zero}) \text{ else } (f \text{ one})) (\lambda x. x)$

No!

Polymorphism

Why is the following expression not well-typed?

$(\lambda f. \text{if } (f \text{ true}) \text{ then } (f \text{ zero}) \text{ else } (f \text{ one})) (\lambda x. x)$

f is applied to `true`

- ▶ Can we assign the type $\mathbb{B} \rightarrow \mathbb{B}$ to $\lambda x. x$?
- ▶ ... no, f is also applied to `zero`

f is applied to `zero`

- ▶ Can we assign the type $\mathbb{N} \rightarrow \mathbb{N}$ to $\lambda x. x$?
- ▶ ... no, f is also applied to `true`

Can we assign the type $\alpha \rightarrow \alpha$ to $\lambda x. x$?

- ▶ the type inference algorithm we saw in lecture 8 would generate:
- ▶ a constraint $\alpha \rightarrow \alpha = \mathbb{B} \rightarrow \mathbb{B}$
- ▶ a constraint $\alpha \rightarrow \alpha = \mathbb{N} \rightarrow T$
- ▶ which would not be solvable because $\mathbb{B} \neq \mathbb{N}$

We cannot use f on both `Ns` and `Bs`

Let-Polymorphism

How could we make the following expression well-typed?

$(\lambda f. \text{if } (f \text{ true}) \text{ then } (f \text{ zero}) \text{ else } (f \text{ one})) (\lambda x. x)$

Using the let-notation:

$\text{let } f = \lambda x. x \text{ in}$
 $\text{if } (f \text{ true}) \text{ then } (f \text{ zero}) \text{ else } (f \text{ one})$

Computationally $\text{let } x = M \text{ in } N$ behaves similar to $(\lambda x. N) M$, but not from a typing perspective.

Can you think of ways to modify this expression to make it type-check?

We could define a $\mathbb{B}$ - f and a $\mathbb{N}$ - f :

$\text{let } f_{\text{bool}} = \lambda x. x \text{ in}$
 $\text{let } f_{\text{nat}} = \lambda x. x \text{ in}$
 $\text{if } (f_{\text{bool}} \text{ true}) \text{ then } (f_{\text{nat}} \text{ zero}) \text{ else } (f_{\text{nat}} \text{ one})$

This is not ideal, as it requires duplicating the code
(violates the **abstraction principle**)

Let-Polymorphism

How could we make the following expression well-typed?

```
let  $f = \lambda x.x$  in  
if ( $f$  true) then ( $f$  zero) else ( $f$  one)
```

Going back to assigning the type $\alpha \rightarrow \alpha$ to f :

- ▶ We would like to instantiate α differently for different uses of f
- ▶ Instead of recording types in type environments, we will record **type schemes** of the form $\forall \alpha_1, \dots, \alpha_n. T$
- ▶ Only for let-expressions
- ▶ A type scheme indicates which variables can be instantiated differently in different uses of an expression of that type

We will

- ▶ assign the type scheme $\sigma = \forall \alpha. \alpha \rightarrow \alpha$ to f
- ▶ instantiate σ with $\mathbb{B}$ when type-checking f 's 1st use
- ▶ instantiate σ with $\mathbb{N}$ when type-checking f 's 2nd use
- ▶ instantiate σ with $\mathbb{N}$ when type-checking f 's 3rd use

Let-Polymorphism

We want the following expression to be well-typed:

```
let  $f = \lambda x.x$  in  
if ( $f$  true) then ( $f$  zero) else ( $f$  one)
```

We will assign the following type scheme σ to f : $\forall \alpha. \alpha \rightarrow \alpha$

- ▶ $\alpha \rightarrow \alpha$ is the type inferred for $\lambda x.x$, e.g., using Wand's algorithm
- ▶ The process of quantifying over type variables is called **generalization**

Why do we have to specify which type variables to generalize?

Consider the following example:

- ▶ Let $M \equiv \text{let } g = f \text{ in } g \text{ zero}$,
- ▶ in an environment Γ_1 where $\Gamma_1(f) = \alpha \rightarrow \alpha$
- ▶ vs an environment Γ_2 where $\Gamma_2(f) = \forall \alpha. \alpha \rightarrow \alpha$.

In the first case, we could provide $f \equiv \lambda x. \text{if } x \text{ then false else true}$, and specialize α to $\mathbb{B}$. But

$M[f \setminus \lambda x. \text{if } x \text{ then false else true}] \equiv \text{if zero then false else true}.$

Let-Polymorphism

What variables can we generalize when typing let-expressions?

Type environments are now of the form:

$$x_1 : \sigma_1, \dots, x_n : \sigma_n$$

Free type variables of schemes and environments:

$$\begin{aligned} \text{FV}(\forall \alpha_1, \dots, \alpha_m. T) &= \text{FV}(T) \setminus \{\alpha_1, \dots, \alpha_m\} \\ \text{FV}(x_1 : \sigma_1, \dots, x_n : \sigma_n) &= \text{FV}(\sigma_1) \cup \dots \cup \text{FV}(\sigma_n) \end{aligned}$$

What variables can we generalize when typing `let  $x = M$  in  $N$` ?

- ▶ let the type environment be Γ
- ▶ let T be the type assigned to M
- ▶ we assign the scheme $\forall \text{FV}(T) \setminus \text{FV}(\Gamma). T$ to x

This form of polymorphism is called **parametric polymorphism**, variants of which are used in languages such as SML, OCaml, Haskell, etc.

Let-Polymorphism

Example 1:

- ▶ given `let f = λx.x in if (f true) then (f zero) else (f one)`
- ▶ where $\Gamma = \cdot$
- ▶ we assign the type scheme $\forall\alpha.\alpha \rightarrow \alpha$ to `f`

We write T for $\forall.T$ (no quantified type variables)

Example 2:

- ▶ given `let g = f in g x`
- ▶ where $\Gamma = x : \alpha, f : \alpha \rightarrow \alpha$
- ▶ we assign the type scheme $\forall.\alpha \rightarrow \alpha$ to `g`

Let-Polymorphism

Typing rules:

We modify the variable rule to allow instantiating schemes:

$$\frac{}{\Gamma, x : \forall \alpha_1, \dots, \alpha_n. T \vdash x : T[\alpha_1 \mapsto T_1, \dots, \alpha_n \mapsto T_n]} \text{VAR}$$

We modify the let rule to allow creating schemes:

$$\frac{\Gamma \vdash M : T \quad \Gamma, x : \forall \text{FV}(T) \setminus \text{FV}(\Gamma). T \vdash N : U}{\Gamma \vdash \text{let } x = M \text{ in } N : U} \text{LET}$$

The other rules do not change

Let-Polymorphism

Provide a derivation that shows that

let $f = \lambda x.x$ in
if (f true) then (f zero) else (f one)

is well-typed

Here is a proof where $\Gamma = f : \forall \alpha. \alpha \rightarrow \alpha$:

$$\frac{\frac{\frac{}{x : \alpha \vdash x : \alpha} \text{VAR}}{\cdot \vdash \lambda x.x : \alpha \rightarrow \alpha} \text{ABS} \quad \frac{\frac{\frac{}{\Gamma \vdash f : \mathbb{B} \rightarrow \mathbb{B}} \text{VAR} \quad \frac{}{\Gamma \vdash \text{true} : \mathbb{B}} \text{T}}{\Gamma \vdash f \text{ true} : \mathbb{B}} \text{APP} \quad \Pi_1 \quad \Pi_2}{\Gamma \vdash \text{if } (f \text{ true}) \text{ then } (f \text{ zero}) \text{ else } (f \text{ one}) : \mathbb{N}} \text{ITE}}{\cdot \vdash \text{let } f = \lambda x.x \text{ in if } (f \text{ true}) \text{ then } (f \text{ zero}) \text{ else } (f \text{ one}) : \mathbb{N}} \text{LET}$$

where Π_1 :

$$\frac{\frac{}{\Gamma \vdash f : \mathbb{N} \rightarrow \mathbb{N}} \text{VAR} \quad \frac{}{\Gamma \vdash \text{zero} : \mathbb{N}} \text{Z}}{\Gamma \vdash f \text{ zero} : \mathbb{N}} \text{APP}$$

and Π_2 (one is succ zero):

$$\frac{\frac{}{\Gamma \vdash f : \mathbb{N} \rightarrow \mathbb{N}} \text{VAR} \quad \frac{\frac{}{\Gamma \vdash \text{zero} : \mathbb{N}} \text{Z}}{\Gamma \vdash \text{succ zero} : \mathbb{N}} \text{S}}{\Gamma \vdash f (\text{succ zero}) : \mathbb{N}} \text{APP}$$

System F

Note that

- ▶ $\text{let } f = \lambda x.x \text{ in if } (f \text{ true}) \text{ then } (f \text{ zero}) \text{ else } (f \text{ one})$ is typable
- ▶ but $(\lambda f.\text{if } (f \text{ true}) \text{ then } (f \text{ zero}) \text{ else } (f \text{ one})) (\lambda x.x)$ is not

Let-polymorphism (or **ML-style polymorphism**) is restrictive: functions are not allowed to take polymorphic values as arguments.

- ▶ introduced by Milner in ML in 1978
- ▶ variants of which have been incorporated in many languages

We will now consider a more powerful notion of polymorphism:
impredicative polymorphism.

Supported by **System F**

- ▶ discovered by Jean-Yves Girard in 1972
- ▶ and by John Reynolds in 1974 (who called it the polymorphic λ -calculus)
- ▶ an extension of the Simply Typed λ -Calculus
- ▶ it allows abstracting and applying types as well as expressions

System F

Syntax (Church-style):

$$\begin{aligned} T &::= \alpha \mid \mathbb{B} \mid \mathbb{N} \mid T \rightarrow T \mid \forall \alpha. T \\ M &::= x \mid \lambda x : T. M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M \\ &\quad \mid \text{let } x = M \text{ in } M \\ &\quad \mid \text{zero} \mid \text{succ } M \mid \text{pred } M \mid \text{iszero } M \\ &\quad \mid \lambda \alpha. M \mid M\{T\} \end{aligned}$$

New forms:

- ▶ universal types: $\forall \alpha. T$
- ▶ type abstractions: $\lambda \alpha. M$
- ▶ type applications: $M\{T\}$

Values: $V ::= \lambda x : T. M \mid \text{true} \mid \text{false} \mid \text{zero} \mid \text{succ } V \mid \lambda \alpha. M$

Call-by-value evaluation contexts

$$\begin{aligned} C &::= \bullet \mid C M \mid V C \mid \text{if } C \text{ then } M \text{ else } M \\ &\quad \mid \text{let } x = C \text{ in } M \mid \text{pred } C \mid \text{iszero } C \mid \text{succ } C \\ &\quad \mid C\{T\} \end{aligned}$$

System F

Call-by-value small-step operational semantics rules:

$$\frac{}{(\lambda x : T.M) V \longrightarrow_v M[x \setminus V]} \beta$$

$$\frac{M \longrightarrow_v N}{C[M] \longrightarrow_v C[N]} \text{CTX}_C$$

$$\frac{}{\text{if true then } M \text{ else } N \longrightarrow_v M} \text{IteT}$$

$$\frac{}{\text{if false then } M \text{ else } N \longrightarrow_v N} \text{IteF}$$

$$\frac{}{\text{let } x = V \text{ in } M \longrightarrow_v M[x \setminus V]} \text{LetV}$$

$$\frac{M \longrightarrow_v N}{\text{succ } M \longrightarrow_v \text{succ } N} \text{Succ}$$

$$\frac{}{\text{pred zero} \longrightarrow_v \text{zero}} \text{PredZ}$$

$$\frac{}{\text{pred (succ } V) \longrightarrow_v V} \text{PredS}$$

$$\frac{}{\text{iszero zero} \longrightarrow_v \text{true}} \text{IsZZ}$$

$$\frac{}{\text{iszero (succ } V) \longrightarrow_v \text{false}} \text{IsZS}$$

$$\frac{}{(\lambda \alpha.M)\{T\} \longrightarrow_v M[\alpha \setminus T]} \text{T}\beta$$

where the capture-avoiding substitution operation on types $M[\alpha \setminus T]$ is similar to the substitution on terms $M[x \setminus N]$, but operates on types and type variables instead. We always rename bound type variables (in type abstractions of the form $\lambda \alpha.M$ and universal types of the form $\forall \alpha.T$) that clash with free type variables before substituting.

System F

What do these expressions reduce to?

- ▶ $(\lambda\alpha.\lambda x : \alpha.x)\{\mathbb{N}\} \longrightarrow_v \lambda x : \mathbb{N}.x$
- ▶ $(\lambda\alpha.\lambda x : \alpha.x)\{\mathbb{N}\} \text{ zero} \longrightarrow_v^* \text{zero}$
- ▶ $(\lambda\alpha.\lambda x : \alpha.x)\{\mathbb{B}\} \longrightarrow_v \lambda x : \mathbb{B}.x$
- ▶ $(\lambda\alpha.\lambda x : \alpha.x)\{\mathbb{B}\} \text{ true} \longrightarrow_v^* \text{true}$
- ▶ $(\lambda\alpha.\lambda x : \alpha.x)\{\mathbb{N} \rightarrow \mathbb{N}\} \longrightarrow_v \lambda x : \mathbb{N} \rightarrow \mathbb{N}.x$
- ▶ $(\lambda\alpha.\lambda x : (\forall\alpha'.\alpha' \rightarrow \alpha).x)\{\alpha' \rightarrow \mathbb{N}\} \longrightarrow_v \lambda x : (\forall\alpha''.\alpha'' \rightarrow \alpha' \rightarrow \mathbb{N}).x$

Note that we renamed the bound α' into α'' in $\forall\alpha'.\alpha' \rightarrow \alpha$ above to avoid capturing the free α' in $\alpha' \rightarrow \mathbb{N}$

System F

Typing rules:

$$\begin{array}{c}
 \frac{}{\Gamma, x : T \vdash x : T} \text{VAR} \qquad \frac{\Gamma, x : T \vdash M : U}{\Gamma \vdash \lambda x : T. M : T \rightarrow U} \text{ABS} \\
 \\
 \frac{\Gamma \vdash M : T \rightarrow U \quad \Gamma \vdash N : T}{\Gamma \vdash M N : U} \text{APP} \\
 \\
 \frac{}{\Gamma \vdash \text{true} : \mathbb{B}} \text{T} \qquad \frac{}{\Gamma \vdash \text{false} : \mathbb{B}} \text{F} \\
 \\
 \frac{\Gamma \vdash M : \mathbb{B} \quad \Gamma \vdash N : T \quad \Gamma \vdash P : T}{\Gamma \vdash \text{if } M \text{ then } N \text{ else } P : T} \text{ITE} \\
 \\
 \frac{}{\Gamma \vdash \text{zero} : \mathbb{N}} \text{Z} \qquad \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \text{succ } M : \mathbb{N}} \text{S} \\
 \\
 \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \text{pred } M : \mathbb{N}} \text{S} \\
 \\
 \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \text{iszero } M : \mathbb{B}} \text{I} \qquad \frac{\Gamma \vdash M : T \quad \Gamma, x : T \vdash N : U}{\Gamma \vdash \text{let } x = M \text{ in } N : U} \text{LET} \\
 \\
 \frac{\Gamma \vdash M : T}{\Gamma \vdash \lambda \alpha. M : \forall \alpha. T} \text{TABS} \qquad \frac{\Gamma \vdash M : \forall \alpha. T}{\Gamma \vdash M \{U\} : T[\alpha \setminus U]} \text{TAPP}
 \end{array}$$

In addition **TABS** requires that $\alpha \notin \text{FV}(\Gamma)$

System F

Provide a derivation of

$(\lambda f : \forall \alpha. \alpha \rightarrow \alpha. \text{if } (f\{\mathbb{B}\} \text{ true}) \text{ then } (f\{\mathbb{N}\} \text{ zero}) \text{ else } (f\{\mathbb{N}\} \text{ one})) (\lambda \alpha. \lambda x : \alpha. x)$

in System F

Here is a proof where $\Gamma = f : \forall \alpha. \alpha \rightarrow \alpha$:

$$\begin{array}{c}
 \frac{}{\Gamma \vdash f : \forall \alpha. \alpha \rightarrow \alpha} \text{VAR} \\
 \frac{}{\Gamma \vdash f\{\mathbb{B}\} : \mathbb{B} \rightarrow \mathbb{B}} \text{TAPP} \quad \frac{}{\Gamma \vdash \text{true} : \mathbb{B}} \text{T} \\
 \hline
 \frac{}{\Gamma \vdash f\{\mathbb{B}\} \text{ true} : \mathbb{B}} \text{APP} \quad \frac{}{\Gamma \vdash \text{if } (f\{\mathbb{B}\} \text{ true}) \text{ then } (f\{\mathbb{N}\} \text{ zero}) \text{ else } (f\{\mathbb{N}\} \text{ one}) : \mathbb{N}} \Pi_2 \quad \frac{}{\Gamma \vdash \text{if } (f\{\mathbb{B}\} \text{ true}) \text{ then } (f\{\mathbb{N}\} \text{ zero}) \text{ else } (f\{\mathbb{N}\} \text{ one}) : \mathbb{N}} \Pi_3 \\
 \hline
 \frac{}{\Gamma \vdash \text{if } (f\{\mathbb{B}\} \text{ true}) \text{ then } (f\{\mathbb{N}\} \text{ zero}) \text{ else } (f\{\mathbb{N}\} \text{ one}) : \mathbb{N}} \text{ITE} \\
 \hline
 \frac{}{\cdot \vdash \lambda f : \forall \alpha. \alpha \rightarrow \alpha. \text{if } (f\{\mathbb{B}\} \text{ true}) \text{ then } (f\{\mathbb{N}\} \text{ zero}) \text{ else } (f\{\mathbb{N}\} \text{ one}) : (\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \mathbb{N}} \text{ABS} \quad \frac{}{\cdot \vdash \lambda x : \alpha. x : \alpha \rightarrow \alpha} \text{VAR} \\
 \hline
 \frac{}{\cdot \vdash \lambda \alpha. \lambda x : \alpha. x : \alpha \rightarrow \alpha} \text{ABS} \quad \frac{}{\cdot \vdash \lambda \alpha. \lambda x : \alpha. x : \alpha \rightarrow \alpha} \text{TABS} \\
 \hline
 \frac{}{\cdot \vdash (\lambda f : \forall \alpha. \alpha \rightarrow \alpha. \text{if } (f\{\mathbb{B}\} \text{ true}) \text{ then } (f\{\mathbb{N}\} \text{ zero}) \text{ else } (f\{\mathbb{N}\} \text{ one})) (\lambda \alpha. \lambda x : \alpha. x) : \mathbb{N}} \text{APP}
 \end{array}$$

where Π_1 (left) and Π_2 (right) are ($\text{one} = \text{succ zero}$):

$$\begin{array}{c}
 \frac{}{\Gamma \vdash f : \forall \alpha. \alpha \rightarrow \alpha} \text{VAR} \\
 \frac{}{\Gamma \vdash f\{\mathbb{N}\} : \mathbb{N} \rightarrow \mathbb{N}} \text{TAPP} \quad \frac{}{\Gamma \vdash \text{zero} : \mathbb{N}} \text{Z} \\
 \hline
 \frac{}{\Gamma \vdash f\{\mathbb{N}\} \text{ zero} : \mathbb{N}} \text{APP}
 \end{array}
 \quad
 \begin{array}{c}
 \frac{}{\Gamma \vdash f : \forall \alpha. \alpha \rightarrow \alpha} \text{VAR} \\
 \frac{}{\Gamma \vdash f\{\mathbb{N}\} : \mathbb{N} \rightarrow \mathbb{N}} \text{TAPP} \quad \frac{}{\Gamma \vdash \text{one} : \mathbb{N}} \text{S} \\
 \hline
 \frac{}{\Gamma \vdash f\{\mathbb{N}\} \text{ one} : \mathbb{N}} \text{APP}
 \end{array}$$

Conclusion

What did we cover today?

- ▶ Polymorphism
- ▶ Let-polymorphism
- ▶ System F

Further reading

- ▶ Type inference is also possible in presence of let-polymorphism
- ▶ Type inference in System F is undecidable
- ▶ “Types and Programming Languages”, Pierce, Chapters 22 & 23

Next time?

- ▶ Abstraction